

Exploiting Causal Provenance for Credit Assignment in Reinforcement Learning for Dynamic Task Assignment: Mechanisms, Guarantees, and Limits

Riccardo Lo Bianco^{a,*}, Co-authors (TODO)^a

^a*Eindhoven University of Technology, Eindhoven, The Netherlands*

Abstract

Dynamic task assignment problems—allocating stochastic streams of tasks to limited, contended resources—are increasingly solved by deep reinforcement learning (DRL) over executable process models such as Action-Evolution Petri Nets (A-E PN). Existing approaches treat the model as a black-box (semi-)Markov decision process and apply generic policy-gradient methods (PPO), discarding the *causal provenance* the Petri net records for free: which past decisions influenced which rewards. We show this provenance can be exploited for credit assignment in two complementary ways. First, s-ccf partitions each episode’s decisions into causal components by union-find over reward provenance and credits every decision only with its own component’s discounted reward-to-go; we prove it is an unbiased, variance-reducing policy-gradient estimator, show that its naive realized-partition predecessor (ccf) is in fact biased at shared-resource handoffs (an exact sign-flip demonstration), and that the fix requires only computing the partition from the static net topology instead. Second, cfpk uses provenance not as a filter but as a replayable causal model: it forks the simulator to obtain an *exact* per-decision counterfactual credit, and we prove a novel cost reduction—coupling truncation—that shares one simulated continuation between the two forked branches whenever they provably reconverge, which we show cancels exactly in the estimator (not merely reduces variance) and measure a 1.37× wall-clock speedup with byte-identical training outcomes. Empirically, s-ccf reduces to PPO when the system forms a single causal component (a shared-bottleneck queue), correctly tying it, and beats PPO decisively as concurrent independence grows, including on a realistic multi-site skills-based routing problem where PPO fails to learn (normalized return 0.15) while s-ccf succeeds (0.52, $p = 0.005$, Cohen’s $d = 1.27$), at essentially the same computational cost. Finally, we investigate whether the honest null both mechanisms predict on a genuinely single-component bottleneck can be closed by any other provenance- or structure-exploiting route, and rule out three independently-motivated candidates with proofs rather than single discouraging runs: potential-based reward shaping (safe by theorem, empirically counter-productive at practical budgets), static structural input features (provably redundant given the network’s existing action-type encoding), and a previously undocumented graph-actor architecture blind spot, which we prove is real via exact-invariance under an untrained network, fix, and show still does not move the bottleneck. This precisely localizes the bottleneck’s remaining difficulty away from credit assignment and observability—a decision rule for practitioners as much as a technical result.

Keywords: Reinforcement learning, Credit assignment, Dynamic task assignment, Petri nets, Decomposition,

1. Introduction

Dynamic task assignment is a core operational problem: a stream of tasks arrives stochastically and must be allocated, over time, to a limited set of contended resources so as to optimize a throughput or cost objective. It underlies scheduling, service-system staffing, manufacturing control, and business-process operation. A recent line of work models such problems with *Action-Evolution Petri Nets* (A-E PN), an executable formalism that unifies Markov decision processes with colored Petri nets, and solves them with deep reinforcement learning (Lo Bianco et al., 2023, 2025b,a). This program has established that A-E PN can *model* a broad taxonomy of assignment problems, that a graph-based representation makes them learnable at scale, and that a software library makes the whole pipeline practical.

In every case, however, the *learning engine* is a generic policy-gradient method—typically Proximal Policy Optimization (PPO) (Schulman et al., 2017)—applied to the induced (semi-)Markov decision process as if it were a black box. This discards structure that the Petri net makes explicit and records for free: the *token provenance*, i.e. which tokens were consumed and produced by each transition firing, and hence which past decisions lie on the causal lineage of each reward.

The problem.. Policy-gradient methods credit each decision with the *return-to-go*: the sum of all rewards occurring after it, minus a learned baseline. In a system where many cases flow concurrently, the return-to-go of a decision about case i includes the rewards of every other case that happens to complete afterwards—rewards the decision had no causal influence over. The critic removes the *mean* of this cross-case reward, but not its *variance*. With K independent cases in flight, a single decision’s advantage carries the reward variance of all K , so the learning signal is buried in concurrent-case noise, and the noise grows with concurrency. This is the multi-entity credit-assignment problem that difference rewards (Wolpert and Tumer, 2001) and value decomposition (Foerster et al., 2018; Sunehag et al., 2018; Rashid et al., 2018) were designed for—but those methods require the entity partition to be specified by hand.

Our approach.. We exploit token provenance to solve this in two complementary ways, spanning a bias/variance/granularity/cost spectrum, and we report a third, equally rigorous investigation into where this kind of exploitation stops paying off. First, for each episode we build the lineage DAG and, for each reward, union the decisions on its causal ancestry into a single *causal component*; each decision is then credited only with its own component’s discounted reward-to-go (s-ccf). In OR terms this is a *decomposition* method—in the lineage of Benders and Dantzig–Wolfe, we decompose

*Corresponding author.

Email address: r.lo.bianco@tue.nl (Riccardo Lo Bianco)

the policy-gradient return along the causal-component structure of the stochastic system—but the decomposition is discovered per episode from provenance rather than imposed a priori. Second, rather than partition anything, *CFPK* forks the simulator at a decision and replays the true environment dynamics under the taken action and an alternative from the identical pre-decision state, obtaining an *exact* counterfactual credit with no partition to get wrong; we show its dominant extra cost can be reduced, exactly and for free, whenever the two forked branches provably reconverge to a common state. Third, having established precisely when these mechanisms help—and, by the same theory, precisely when they collapse to a no-op—we ask whether the residual gap on a single-component bottleneck can be closed by any other route that only changes what the learner observes, and show, with proofs rather than speculation, that three well-motivated candidates cannot.

Contributions..

1. We identify that generic policy-gradient RL discards the provenance structure A-E PN makes explicit, and that this causes credit-assignment variance that grows with concurrent independence (Section 4).
2. *s-ccf*: a structure-*derived* (not hand-specified) factored credit method, computed by union-find over the *static* net topology. We prove it is unbiased and variance-reducing (Propositions 1– 2, Corollary 1), and give an exact sign-flip demonstration that its naive realized-partition predecessor *ccf* is biased at shared-resource handoffs, which *s-ccf* fixes by construction (Proposition 3; Sections 4, 5, 6).
3. *CFPK*: provenance used as a replayable causal model for *exact* counterfactual credit, with a novel, provably lossless cost reduction (coupling truncation, Proposition 4) measured at a 1.37× wall-clock speedup with byte-identical training outcomes (Sections 4, 5, 6).
4. Empirically, *s-ccf* reduces exactly to PPO on single-component problems and beats it decisively as independence grows, including on a realistic multi-site skills-based routing problem, at $\sim 1\times$ the compute (Section 6).
5. A rigorous characterization of the *boundary* of provenance exploitation: we stress-test three independent, well-motivated alternative routes to closing the honest null both mechanisms predict on a single-component bottleneck—denser reward signal, richer structural input features, and a more observant graph-actor architecture—and show each is either provably inert or empirically neutral, via proofs and exact-invariance tests rather than single discouraging runs (Section 7). This localizes the bottleneck’s remaining difficulty away from credit assignment and observability, giving practitioners a decision rule (Section 8) rather than an unconditional claim.

2. Related work

DRL for dynamic task assignment and operational control.. A growing literature applies DRL to scheduling, routing, and business-process decision-making, typically encoding the problem instance as a graph consumed by a graph neural network policy: greedy graph-embedding heuristics for combinatorial construction (Dai et al., 2017), attention-based routing policies (Nazari et al., 2018), and GNN-based dispatching for job-shop scheduling (Zhang et al., 2020), the

last especially close in spirit to our setting. Mazyavkina et al. (2021) survey this broader RL-for- combinatorial- optimization literature; Bengio et al. (2021), published in this journal, give the methodological tour d’horizon our OR framing draws on most directly. Within the A-E PN program, Lo Bianco et al. (2023) introduce the formalism and a taxonomy of assignment archetypes, Lo Bianco et al. (2025b) give a graph representation for infinite state/action spaces, and Lo Bianco et al. (2025a) provide a library supporting partial observability and multiple decisions. Across this literature, the learning algorithm is almost universally a black box on top of the graph encoding; the simulator’s own execution trace—which decisions caused which outcomes—is not itself treated as a source of learning signal. Our contribution is orthogonal to and compatible with all of the above: it is a credit-assignment mechanism, not a new policy architecture or problem encoding.

Credit assignment in RL. Return decomposition (Arjona-Medina et al., 2019) learns to redistribute return along a trajectory via a return-predicting model’s contribution analysis; Hindsight Credit Assignment (Harutyunyan et al., 2019) reweights past actions by their learned likelihood of having caused an observed outcome. Both are *learned* redistribution mechanisms trained on correlational patterns across trajectories. Our mechanisms differ in kind: token provenance in an A-E PN is not inferred, it is recorded exactly by the simulator, so s-ccf and cfpk’s credit signals carry no approximation error from the attribution step itself. This is close in motivation to model-free counterfactual credit assignment (Mesnard et al., 2021), which conditions value functions on future events to build low-variance counterfactual baselines; cfpk’s simulator replay can be read as the model-based analogue of the same goal, made exact because the simulator is itself an available, cheap-to-fork causal model, not something that needs to be learned.

Factored / multi-agent credit. Difference rewards (Wolpert and Tumer, 2001) credit each agent with its marginal contribution relative to a counterfactual baseline; COMA (Foerster et al., 2018) learns this baseline via a centralized critic; value decomposition (Sunehag et al., 2018; Rashid et al., 2018) factors a joint value across agents under additivity or monotonicity constraints. All require the entity/agent partition to be given. s-ccf needs no such specification: the causal-component partition is derived automatically, per episode, from token provenance, and can vary in shape as the realized case mix changes—a single-agent problem (one policy, many concurrent cases) is factored the same way a genuinely multi-agent one would be.

Potential-based reward shaping. Ng et al. (1999) prove that adding a potential-based term to every step’s reward cannot change an MDP’s optimal policy, for any potential function. We use the SMDP-discounted generalization of this theorem not as a headline mechanism but as one of three falsified alternative routes in our boundary-condition investigation (Section 7): a theorem-safe, empirically counter-productive result at practical training budgets, included because it isolates sample-efficiency effects cleanly from the bias effects that are our main concern in Section 5.

Graph neural networks for combinatorial optimization and heterogeneous actor-critic architectures. GNNs are now a standard state encoder for RL over combinatorial structures with variable size and topology (Dai et al., 2017; Zhang et al., 2020); our environments use a heterogeneous variant, the Heterogeneous Graph Transformer (Hu et al., 2020),

encoding places and transitions as distinct node types. Section 7 documents an architectural finding of independent interest: a per-node actor decoder combined with directed-only (non-reversed) message-passing edges can be *exactly* blind to state outside its forward-reachable neighborhood, a limitation the critic (which pools) does not share. To our knowledge this is not previously documented for heterogeneous actor-critic graph architectures, and the argument is orthogonal to the specific choice of GNN backbone.

Decomposition in operations research. Dantzig–Wolfe decomposition (Dantzig and Wolfe, 1960) and Benders decomposition (Benders, 1962) break a hard, structured optimization problem into a master problem and weakly-coupled subproblems along a hand-identified block structure. s-ccf applies the same idea to the policy-gradient *return*: decompose it along a block structure derived automatically, per episode, from the simulator’s own causal record, rather than a static constraint matrix’s block structure identified a priori.

3. Background

Because Action-Evolution Petri Nets are not yet widely known, we describe the formalism intuitively before defining the learning problem.

Petri nets and colored tokens. A Petri net is a bipartite directed graph of *places* (drawn as circles) and *transitions* (drawn as bars), connected by arcs. Places hold *tokens*; in a *colored* net each token carries typed data—a task’s type, a resource’s identity. A transition is *enabled* when its input places jointly hold a matching combination of tokens, called a *binding*; *firing* the transition consumes that binding and produces new tokens in its output places. The distribution of tokens over places—the *marking*—is the system state, and it evolves as transitions fire.

The action/evolution split. An Action-Evolution Petri Net (A-E PN) (Lo Bianco et al., 2023) partitions the transitions into two kinds and adds a global clock. *Evolution* transitions model the exogenous dynamics: they fire automatically according to stochastic, timed rules—task arrivals, service completions—advancing the clock. *Action* transitions model the controllable decisions: whenever several action bindings are simultaneously enabled, an external *policy* selects which one to fire. Selected firings may carry a scalar *reward*. Figure 1 shows a minimal assignment net: a waiting task and a free resource are consumed by an *assign* action transition—the decision of which task to serve with which resource—producing a *busy* token; an evolution transition *complete* later frees the resource (returning it to the pool) and emits a reward.

The induced SMDP. Executing an A-E PN therefore alternates between the environment evolving the marking (evolution firings that advance the clock) and the agent choosing among enabled action bindings—exactly a semi-Markov decision process (SMDP). We write s_d for the marking at decision d , a_d for the chosen binding, and u_d for its clock; delayed rewards are discounted by the SMDP factor $\rho_d(t) = e^{-\beta \max(0, t-u_d)}$. Dynamic task assignment problems—matching stochastic task streams to contended resources—map onto this template directly (Lo Bianco et al., 2023,

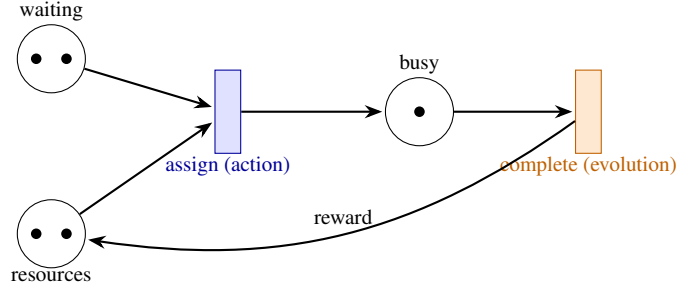


Figure 1: A minimal A-E PN for task assignment. Circles are places holding colored tokens; bars are transitions. The *assign action* transition (blue) is a decision point: the policy chooses which waiting task to bind to which free resource. The *complete evolution* transition (orange) fires automatically after a stochastic delay, returns the resource, and emits a reward.

2025b,a). Because the simulator is a deterministic function of the marking, the clock, and the (recorded) exogenous random draws, it can be *snapshotted and re-run* from any decision point under an alternative action—the capability cFPK’s forking (Section 4) relies on, and a further consequence of the same executable-model property that makes A-E PN a unified modeling-and-solving framework in the first place.

Policy gradient over A-E PN. The policy π_θ is a graph neural network over the assignment-graph representation (Lo Bianco et al., 2025b), trained by PPO (Schulman et al., 2017) with generalized advantage estimation (Schulman et al., 2016). The canonical unbiased gradient estimator credits each decision with its reward-to-go against a learned baseline $V(s_d)$; both mechanisms below modify only how that reward-to-go is computed.

4. Method: two mechanisms, two affordances of the formalism

Two affordances, one formalism.. An A-E PN gives a learner two things a black-box SMDP does not, and our two mechanisms exploit one each. The first is *token provenance*: a causal ancestry recorded for free by the firing semantics, which s-ccf/ccf use to partition credit (Section 4.1). The second is that the net is a *perfect, resettable forward model*: it can be snapshotted, forked and replayed exactly, which cFPK uses to measure a counterfactual directly (Section 4.2). We keep them separate deliberately. cFPK consults no provenance DAG at all—its coupling truncation is justified by a state fingerprint (canonical marking plus clock), not by causal history—and an ablation that *adds* a lineage restriction to it is slightly worse on the bottleneck environment. The two are complementary rather than variants of one idea, and Section 6 shows the complementarity is in their coverage: provenance partitioning pays exactly where the problem decomposes, and simulator replay pays where it does not.

Token provenance: the structure s-ccf and ccf exploit.. Each time a transition fires we record which tokens it consumed and which it produced; linking every produced token to the firing and the inputs that created it yields the *token-provenance DAG* (Figure 2). Because in an A-E PN every reward is a deterministic function of the tokens consumed at its firing, this DAG contains the *complete causal ancestry of every reward*: walking backward from a reward

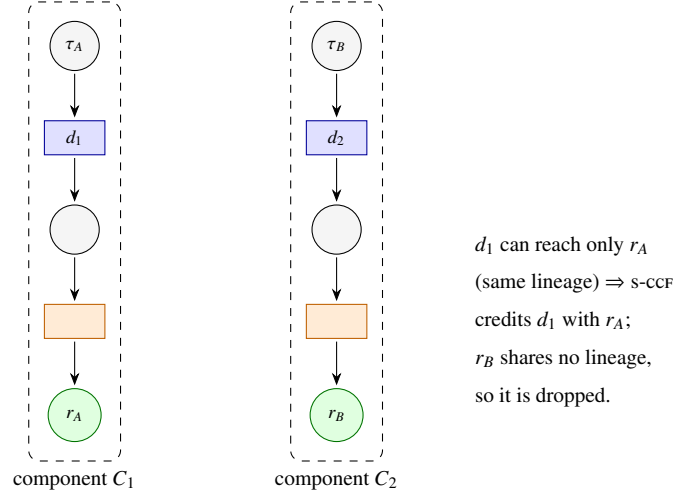


Figure 2: Token provenance and causal components. Two cases flow through disjoint resources, so their provenance never crosses: circles are tokens (τ tasks, r rewards), boxes are firings (blue = action decision d_i , orange = evolution). Union-find over reward ancestries yields two components. A decision’s action can influence only rewards on its own lineage, so s-ccf credits d_1 with r_A and drops the causally-unrelated r_B —whereas the full reward-to-go (PPO) credits d_1 with $r_A + r_B$, injecting the variance of the unrelated case.

reaches exactly the tokens—and hence the action-decisions—that could have influenced it. Formally, for reward j let $\mathcal{A}(j) \subseteq \{1, \dots, D\}$ be the set of action-decisions reachable backward from j in the DAG. This object requires no additional modeling: it is implied by the net’s firing semantics and recorded at negligible cost during simulation. Prior A-E PN learning treats the net as a black-box SMDP and never consults it.

4.1. s-ccf: causal-component-factored credit

A spectrum of credit signals.. Every redistribution scheme differs only in *which rewards each decision is allowed to see* (Table 1). The Monte-Carlo return mc-q credits a decision with *all* future rewards; it equals the PPO reward-to-go and is unbiased but high-variance. A single-lineage restriction (LRQ) credits only rewards on the decision’s own token lineage; this cuts variance but is *biased*, because it discards the opportunity-cost rewards that sibling lineages—the roads not taken—would have earned. ccf keeps the whole *realized* causal component, which fixes LRQ’s opportunity-cost bias but, as Section 5 shows, introduces a different bias of its own at shared-resource handoffs. s-ccf keeps the whole *static* causal component and is unbiased unconditionally.

Causal components via union-find.. For each reward j we collect the decisions in its lineage, $\mathcal{A}(j)$, by a backward walk of the provenance DAG, and union them: two decisions are placed in the same component whenever they jointly caused a common reward. After processing all rewards, the decisions are partitioned into causal components $C_1, \dots, C_M$; each reward with $\mathcal{A}(j) \neq \emptyset$ belongs to a well-defined component $c(j)$. In a multi-copy system with disjoint resources this yields M separate components; when a shared resource links every case, freed resources re-enter the DAG and $M = 1$.

Table 1: The credit spectrum.

scheme	a decision sees. . .	bias	variance
MC-Q	all future rewards (= PPO)	none	high
LRQ	its own token lineage	biased (opportunity cost)	low
CCF	its own <i>realized</i> causal component	biased (joins)	low
s-CCF	its own <i>static</i> causal component	none	low–moderate
CFPK	exact forked counterfactual (no partition)	none	low (cost-bounded)

Component return-to-go. Each decision d (in component $c(d)$, at clock u_d) is credited with the discounted rewards of its own component that occur at or after u_d :

$$G_d^{\text{CCF}} = \sum_{\substack{j: t_j \geq u_d \\ c(j)=c(d)}} \rho_d(t_j) r_j, \quad G_d^{\text{MC-Q}} = \sum_{j: t_j \geq u_d} \rho_d(t_j) r_j, \quad (1)$$

and the advantage is $A_d = G_d^{\text{CCF}} - V(s_d)$. The dropped quantity $\Delta_d = G_d^{\text{MC-Q}} - G_d^{\text{CCF}}$ is the cross-component future reward. Algorithm 1 summarizes the computation; its cost is $O(\text{decisions} \times \text{rewards})$ per episode, negligible next to the GNN forward/backward passes. Postpone (wait) decisions, which under token-flow-off have no reward-causing descendants, are handled by the standard SMDP temporal- difference advantage rather than by this redistribution.

Action-dependent membership and the static fix (s-ccf). The union-find above partitions decisions from the *realized* trace, and that partition can itself depend on the action taken. At an AND-join, a decision’s choice determines whether it merges with a joined decision’s component, so a co-decision’s private reward enters its credit under one action but not the other—even though that reward fires under both. The membership of ccf’s components is then action-dependent, which (Section 5) breaks the assumption behind Proposition 1: ccf becomes biased at joins and can prefer a suboptimal action (Section 6 demonstrates an exact sign flip). We therefore compute the component partition from the *static net topology* instead: two decisions share a component iff they *could* co-cause a reward under *some* action sequence (topological reachability, taking the union over a decision’s competing actions). The resulting estimator, s-ccf, credits each decision with the realized rewards of its *static* component. Its membership is action-invariant by construction, so it is unbiased without any assumption on the realized trajectory (Proposition 3), while retaining ccf’s variance reduction on statically-independent structure. On pure dynamic task assignment the two coincide (assignment decisions create no joins); they diverge only at synchronization transitions.

Which one is the method. It is tempting to present s-ccf alone, since it is the variant with the unconditional guarantee. That would misdescribe what the two estimators do. The static partition buys action-invariance by being conservative, and on loosely-coupled problems it is conservative to the point of being *inert*: whenever any resource could in principle route anywhere, $K_{\text{stat}} = 1$ and s-ccf reduces to PPO exactly—which is the situation on both of our non-constructed

Algorithm 1 Causal-component-factored credit (ccf / s-ccf, differing only in line 4)

```
1: input: episode trace with provenance DAG; decisions  $1..D$ ; rewards  $\{(r_j, t_j)\}$ 
2: initialize union-find over  $\{1, \dots, D\}$ 
3: for each reward  $j$  with  $r_j \neq 0$  do
4:    $\mathcal{A}(j) \leftarrow$  decisions reachable backward from  $j$ 's inputs in the DAG (ccf: realized trace only; s-ccf: static topology, union over competing actions)
5:   union all decisions in  $\mathcal{A}(j)$ 
6: end for
7: for each decision  $d$  do
8:    $G_d \leftarrow \sum_{j: c(j)=c(d), t_j \geq u_d} e^{-\beta(t_j - u_d)} r_j$ 
9: end for
10: return advantages  $A_d = G_d - V(s_d)$ 
```

problems (multi-site routing and scaled assignment; Section 8). ccf is the estimator that acts there, and it is the one carrying the empirical results of Section 6.

We therefore present the two as a *pair* spanning a bias/reach trade-off, selected by the diagnostic of Section 8: s-ccf where the static partition already separates the problem, ccf where only the realized one does. The join bias is not hidden by this framing—it is the price of the second column, we quantify it exactly (Table 4), and we report that we could not induce it in training across four purpose-built configurations.

4.2. CFPK: simulator-replay counterfactual credit and coupling truncation

The estimators above *restrict* which realized rewards count toward a decision's credit—a partition of a single trajectory. CFPK takes a different route: it does not partition anything. At a decision d it forks the simulator and replays the true environment dynamics forward under the taken action a_d and, separately, under an alternative a'_d , from the identical pre-decision state—an exact, model-based counterfactual, since the “model” is the real simulator (Section 3), not a learned approximation.

Exact counterfactual credit.. For a branch $\omega \in \{a_d, a'_d\}$, let $G_d(\omega) = \sum_{j: t_j \geq u_d} \rho_d(t_j) r_j^{(\omega)}$ be the SMDP-discounted return realized by that branch's simulated continuation (policy π_θ thereafter, to a lookahead horizon L or episode end, optionally closed off by a learned value-tail bootstrap at the horizon). The credit signal is $I_d = G_d(a_d) - G_d(a'_d)$: the exact realized counterfactual advantage of the taken action, requiring no lineage partition and hence immune to the shared-handoff bias that motivated s-ccf.

Coupling truncation.. CFPK's dominant cost is the fork itself: a second full simulated continuation per decision. We reduce this, exactly and for free, whenever the two branches' simulated states provably *reconverge*. Simulate both branches one environment step at a time in lockstep, and let $\phi(\cdot)$ be a canonical, symmetry-collapsed (marking,

Algorithm 2 Coupling-truncated paired rollout (core of CFPK)

```
1: input: snapshot at decision  $d$ ; taken action  $a_d$ ; alternative  $a'_d$ 
2: fire  $a_d$  and  $a'_d$  from the snapshot, giving branches  $A, B$  with running discounted totals  $P_A, P_B$ 
3: while both  $A, B$  active and within the lookahead do
4:   if  $\phi(A) = \phi(B)$  then                                      $\triangleright$  reconvergence: same canonical (marking, clock)
5:     simulate ONE continuation  $\tilde{V}$  from the shared state
6:     return  $P_A + \tilde{V}, P_B + \tilde{V}$                                  $\triangleright \tilde{V}$  cancels exactly in the difference
7:   end if
8:   step  $A$  and  $B$  forward one environment step each, updating  $P_A, P_B$ 
9: end while
10: return  $P_A, P_B$  (no reconvergence: exact, untruncated two-branch rollout)
```

clock) state fingerprint (reused from an MCTS transposition-table mechanism). If at round σ the two branches' states coincide, $\phi(s^{(a_d)}(\sigma)) = \phi(s^{(a'_d)}(\sigma)) =: s^*$, decompose each branch's return around s^* : $G_d(a_d) = P_{a_d} + \rho_d(t^*)\tilde{V}(s^*)$, $G_d(a'_d) = P_{a'_d} + \rho_d(t^*)\tilde{V}(s^*)$, where P_ω is branch ω 's own pre-reconvergence realized reward and $\tilde{V}(s^*)$ is a *single* simulated continuation from s^* , reused for both branches rather than drawn twice. Algorithm 2 gives the lockstep rollout; Proposition 4 (Section 5) proves the shared term cancels exactly in I_d , so this is a strict cost reduction with no correctness cost. If the branches never reconverge within the lookahead, the mechanism falls back to two independent continuations—the untruncated, always-exact base case.

4.3. Positioning

Table 1 already positions the component-factored estimators and CFPK on the same bias/variance axis as MC-Q/LRQ; the remaining axes are *reach* and *cost*.

On reach, the two component-factored variants differ only in which partition they use, and the diagnostic counts of Section 8 say when each has anything to work with: s-ccf acts iff $K_{\text{stat}} > 1$, ccf iff $K_{\text{real}} > 1$, and $K_{\text{real}} \geq K_{\text{stat}}$ always. On cost, both are near-free—union-find and component return-to-go are negligible next to the GNN passes—whereas CFPK is exact and fine-grained at *every* decision, including synchronization points where neither partition helps, at a bounded extra cost from forking, one proven and free chunk of which coupling truncation removes.

This gives three regimes rather than two, and Section 8 turns them into a decision rule: use s-ccf when the static partition already separates the problem, ccf when only the realized one does, and CFPK when neither does but the extra simulation budget is available.

5. Theoretical analysis

5.1. s-ccf: unbiasedness and variance reduction

We take as given that the full reward-to-go estimator is unbiased, $\mathbb{E}[\hat{g}^{\text{MC-Q}}] = \nabla_{\theta} J(\theta)$ (Sutton et al., 2000); all statements are relative to it. Let $\mathcal{F}_d$ denote the history up to but excluding a_d .

Assumption 1 (Component mean-exogeneity). *For every decision d , $\mathbb{E}[\Delta_d | \mathcal{F}_d, a_d] = \mathbb{E}[\Delta_d | \mathcal{F}_d]$.*

Assumption 1 holds by A-E PN semantics: a reward is a deterministic function of the tokens it consumes, whose full history is recorded, so a_d can influence r_j only if some descendant of a_d lies on j 's lineage—i.e. only if $c(j) = c(d)$. Cross-component rewards therefore have no causal path from a_d ; the only residual channel is the exogenous randomness driving other components' events, which is independent of a_d under the standard model of independent per-firing draws.

Proposition 1 (Unbiasedness). *Under Assumption 1, $\mathbb{E}[\hat{g}^{\text{CCF}}] = \mathbb{E}[\hat{g}^{\text{MC-Q}}] = \nabla_{\theta} J(\theta)$.*

Proof. The estimators differ only through Δ_d : $\hat{g}^{\text{MC-Q}} - \hat{g}^{\text{CCF}} = \sum_d \nabla_{\theta} \log \pi_{\theta}(a_d | s_d) \Delta_d$. Conditioning on $\mathcal{F}_d$ and drawing $a_d \sim \pi_{\theta}(\cdot | s_d)$,

$$\mathbb{E}[\nabla \log \pi_{\theta}(a_d | s_d) \Delta_d | \mathcal{F}_d] = \mathbb{E}_{a_d}[\nabla \log \pi_{\theta}(a_d | s_d) \mathbb{E}[\Delta_d | \mathcal{F}_d, a_d]].$$

By Assumption 1 the inner expectation equals $\bar{\Delta}_d := \mathbb{E}[\Delta_d | \mathcal{F}_d]$, independent of a_d , so the expression is $\bar{\Delta}_d \mathbb{E}_{a_d}[\nabla \log \pi_{\theta}(a_d | s_d)] = \bar{\Delta}_d \nabla_{\theta} \sum_a \pi_{\theta}(a | s_d) = 0$. Summing over d gives $\mathbb{E}[\hat{g}^{\text{MC-Q}} - \hat{g}^{\text{CCF}}] = 0$. $\square$

Proposition 2 (Variance reduction). *Suppose at decision d the future reward-to-go splits across K causally- independent components, $G_d^{\text{MC-Q}} = \sum_{k=1}^K R_k$ with d in component 1 and $G_d^{\text{CCF}} = R_1$, and that conditional on s_d the R_k are mutually independent with variances σ_k^2 . Then, for the mean baseline $V(s_d) = \mathbb{E}[G_d^{\text{MC-Q}} | s_d]$,*

$$\text{Var}(G_d^{\text{MC-Q}} - V | s_d) = \sum_{k=1}^K \sigma_k^2, \quad \text{Var}(G_d^{\text{CCF}} - V | s_d) = \sigma_1^2.$$

If $\sigma_k^2 \approx \sigma^2$, ccf reduces per-decision advantage variance by a factor $\approx K$ (gradient-noise standard deviation by $\approx \sqrt{K}$).

Proof. Immediate from independence: $\text{Var}(\sum_k R_k) = \sum_k \sigma_k^2$; subtracting the s_d -measurable constant V leaves variances unchanged; and $G_d^{\text{CCF}} = R_1$. $\square$

Corollary 1 (Monotone variance reduction). *Assume only that, conditional on s_d , G_d^{CCF} and Δ_d are uncorrelated (implied by causal independence of distinct components). Then for any baseline $V(s_d)$,*

$$\text{Var}(G_d^{\text{MC-Q}} - V | s_d) = \text{Var}(G_d^{\text{CCF}} - V | s_d) + \text{Var}(\Delta_d | s_d) \geq \text{Var}(G_d^{\text{CCF}} - V | s_d),$$

with equality iff $\text{Var}(\Delta_d | s_d) = 0$ (in particular whenever $M = 1$).

Proof. Write $G_d^{\text{MC-Q}} - V = (G_d^{\text{CCF}} - V) + \Delta_d$; expand the variance; the cross term is $\text{Cov}(G_d^{\text{CCF}}, \Delta_d \mid s_d) = 0$ by assumption. Nonnegativity of $\text{Var}(\Delta_d)$ gives the inequality. $\square$

Corollary 1 is the robust statement: ccf is *never worse* than PPO in advantage variance, for any critic and with no assumption on the number of components, and strictly better wherever cross-component reward exists. When a shared resource collapses the system to one component ($M = 1$), $\Delta_d \equiv 0$ and ccf is identically PPO—the honest null we confirm empirically and build on directly in Section 7.

When Assumption 1 fails, and s-ccf. Assumption 1 treats the component partition as fixed, but ccf reads it from the realized trace, and the partition can depend on a_d . At an AND-join, action A fires the join and merges d with a co-decision d' (so d' 's private reward $r' \in c(d)$), while action B does not ($r' \notin c(d)$); yet r' fires under both. Hence $\mathbb{E}[\Delta_d \mid \mathcal{F}_d, a_d]$ depends on a_d : Assumption 1 fails, and with $r' > r_1 - r_{\text{join}}$ the sign of d 's credit difference reverses, so ccf prefers the suboptimal join (Section 6 gives an exact numerical instance). The remedy is to make membership action-invariant.

Assumption 2 (Static membership). *The component partition is a function of the net topology alone: two decisions share a component iff they can co-cause a reward under some action sequence. In particular it does not depend on a_d .*

Proposition 3 (Unbiasedness of s-ccf). *Under Assumption 2, the rewards s-ccf drops ($\{r_j : c(j) \neq c(d)\}$) are, by topological construction, unreachable from a_d under any action, hence action-independent given s_d . The argument of Proposition 1 then applies verbatim with Assumption 2 in place of Assumption 1: $\mathbb{E}[\hat{g}^{\text{s-ccf}}] = \nabla_\theta J(\theta)$, with no assumption on the realized trajectory.*

s-ccf thus *dominates* ccf: unbiased in the join and shared-handoff regimes where ccf (and LRQ) can flip to a suboptimal action, and identical to ccf on pure task assignment, where assignment decisions create no joins. We establish the bias and the fix at the estimator level (Section 6); a clean training-level demonstration of the ccf flip is elusive, because the very structure that triggers it—a private reward coupled through a join—is an intrinsically hard credit-assignment problem in which even the unbiased baseline (PPO) fails to reach the optimum.

5.2. CFPK: coupling truncation is exact

Assumption 3 (Fingerprint sufficiency). *For any two simulated states s, s' with $\phi(s) = \phi(s')$, the conditional law of all future (policy and exogenous) randomness given s equals that given s' : ϕ is a bisimulation-consistent sufficient statistic for the SMDP's transition-and-reward kernel.*

Assumption 3 holds by construction: the AEPN's dynamics depend on the marking and the clock alone, and ϕ discards nothing but token identities within a class of tokens that are, by construction, interchangeable in every downstream rule (same place, same attributes, same enabled transitions).

Proposition 4 (Coupling truncation is exact). *Under Assumption 3, for any realization of the pre-reconvergence randomness, the coupling-truncated credit signal satisfies $I_d^{\text{coupled}} = G_d(a_d) - G_d(a'_d) = P_{a_d} - P_{a'_d}$ exactly—the shared continuation $\tilde{V}(s^*)$ cancels identically, regardless of its realized value. Consequently I_d^{coupled} targets the same quantity as the naive estimator that draws two independent continuations from s^* , and has strictly lower variance whenever $\text{Var}(\tilde{V}(s^*) | s^*) > 0$: the reconverged tail contributes exactly zero variance to I_d^{coupled} , versus $\rho_d(t^*)^2 \cdot 2 \text{Var}(\tilde{V}(s^*) | s^*)$ (two independent draws) to the naive estimator.*

Proof. Exact cancellation. Immediate from the decomposition: the $\rho_d(t^*)\tilde{V}(s^*)$ term is the same real number added to both $G_d(a_d)$ and $G_d(a'_d)$, so it cancels in the difference for every realization—no appeal to Assumption 3 is needed for this half, only that the implementation reuses one draw. *Same target.* Assumption 3 gives that both branches’ post- σ continuations are drawn from the identical distribution with mean $V^\pi(s^*)$, so the naive estimator’s cross term has expectation $\rho_d(t^*)(V^\pi(s^*) - V^\pi(s^*)) = 0$: both estimators target $\mathbb{E}[P_{a_d} - P_{a'_d}]$. *Variance.* The naive estimator’s two continuation draws are simulated independently of each other and of the pre- σ randomness (Markov property), so its variance is $\text{Var}(P_{a_d} - P_{a'_d}) + \rho_d(t^*)^2 \cdot 2 \text{Var}(\tilde{V}(s^*) | s^*)$, while $I_d^{\text{coupled}} = P_{a_d} - P_{a'_d}$ exactly, giving $\text{Var}(I_d^{\text{coupled}}) = \text{Var}(P_{a_d} - P_{a'_d})$ —strictly smaller whenever the reconverged tail has any variance at all. $\square$

This is a stronger claim than a typical common-random-numbers coupling argument, which *reduces* shared-tail variance by inducing positive correlation between two draws; here the shared draw is not merely correlated across branches, it *is* the same draw, so the tail’s contribution to the paired difference is not reduced but eliminated. Unlike CRN, this needs no alignment of the two branches’ random-number-stream *positions* at reconvergence—only that they have reached the same *state*, which Assumption 3 guarantees determines the same future distribution regardless of how each branch got there. (*Full remarks—no-RNG-alignment detail, agnosticism to the value-tail-bootstrap procedure, the lineage-mode scope gate—in the appendix.*)

6. Experiments

We answer six questions. **Q1:** Does s-ccf reduce to PPO when the system is a single causal component (correctness)? **Q2:** Does s-ccf beat PPO as concurrent independence grows? **Q3:** Does the win hold on a realistic operational problem? **Q4:** Is the win attributable to the *component factoring* specifically, rather than to the SMDP return-to-go machinery it is built on? **Q5:** Does ccf’s bias mechanism manifest concretely, and does s-ccf fix it? **Q6:** Is coupling truncation exact, and does it measurably reduce CFPK’s cost, at real training scale?

Protocol. The baseline is PPO with SMDP-matched discounting, so the *only* difference from s-ccf/ccf is the credit factoring. Returns are normalized so that 0 is a random policy and 1 a strong domain heuristic. We report means with 95% confidence intervals over independent seeds and test significance with Welch’s t and Mann–Whitney U .

Table 2: Multi-site skills-based routing (independent config, 4 sites, 12 seeds, 40 epochs). Normalized greedy return.

method	norm. final (95% CI)	median	seeds > 0.5
PPO (discount-matched)	0.146 $\pm$ 0.150	0.058	2/12
mc-q (lineage ablation)	0.036 $\pm$ 0.121	0.068	0/12
ccf (ours)	0.518 $\pm$ 0.181	0.546	6/12

The lineage ablation (Q4). PPO is the right baseline for “does credit factoring help”, but it cannot separate the *factoring* from the per-decision SMDP return-to-go the factored schemes are built on. We therefore add mc-q as an explicit *lineage ablation*: the identical estimator with the causal-component test deleted, so each decision receives the full discounted return-to-go from its own clock. ccf minus mc-q isolates the contribution of the decomposition itself, and is the comparison we report alongside ccf minus PPO throughout.

Mechanism (Q2, Figure 5). On N causally-independent copies of a base assignment task, the divergence between ccf and PPO credit, $|ccf - mc-q|/scale$, rises monotonically from 0.09 ($N=1$) to 0.84 ($N=8$): more independent components means more concurrent-case noise for ccf to factor away, exactly as Proposition 2 predicts.

Scaling (Q1–Q2, Figure 4). Training on the same family, ccf and PPO are statistically tied at $N=1$ (one component, nothing to factor—the required null), while for $N > 1$ ccf holds near the optimum and PPO degrades and destabilizes (its seed-to-seed variance balloons; individual seeds finish at or below the random baseline). ccf exhibits *stable optimality* where PPO exhibits *credit-collapse*. (These topologies have no shared-resource handoff, so ccf and s-ccf coincide exactly here—see the note following Table 4.)

Realistic operational problem (Q3, Figure 3). We model multi-site skills-based routing: n_{sites} service sites, each with its own arrival stream and a heterogeneous pool of local specialists, plus a shared pool of flexible generalists—the classic resource pooling/flexibility setting (Jordan and Graves, 1995). In the independent-site configuration ($n_{sites}=4$, 12 seeds, 40 epochs), PPO *fails to learn* the routing: its learning curve is flat and it plateaus at normalized return 0.15 (median 0.06; 2/12 seeds exceed 0.5). ccf learns it, rising to 0.52 (median 0.55; 6/12 seeds exceed 0.5). The difference is significant (Welch $p = 0.005$, Mann–Whitney $p = 0.004$) with a large effect size (Cohen’s $d = 1.27$) and disjoint 95% CIs (Table 2).

The win is the factoring, not the return-to-go (Q4, Table 3). Running the mc-q lineage ablation on both families settles what PPO comparisons cannot. On the N -copies sweep the ablation tracks ccf while independence is scarce and then falls away from it: at $N=8$, mc-q is statistically indistinguishable from PPO ($+0.005$, $p = 0.97$) while ccf is $+0.564$ above mc-q (12/12 seeds, $p = 0.0001$). In other words the *entire* margin over PPO at $N=8$ is the component decomposition; the SMDP return-to-go contributes none of it. Multi-site routing replicates this on a non-constructed problem: mc-q reaches 0.036, *below* PPO’s 0.146, while ccf is $+0.482$ above mc-q (11/12 seeds, $p = 0.0005$).

Table 3: Lineage ablation (Q4). mc-q is ccf with the causal-component test deleted, so $\text{ccf} - \text{mc-q}$ isolates the decomposition. Normalized greedy return; 12 seeds throughout; p from a paired t -test on shared seeds.

problem	K	PPO	MC-Q	CCF	CCF $-$ MC-Q	
N -copies, $N=1$	1	0.864	0.895	0.893	-0.003	$(p=0.90)$
N -copies, $N=2$	2	0.319	0.517	0.654	$+0.137$	$(p=0.56)$
N -copies, $N=4$	4	0.523	0.692	0.796	$+0.105$	$(p=0.43)$
N -copies, $N=8$	8	0.362	0.367	0.931	+0.564	$(p=0.0001)$
multi-site routing	$12^\dagger$	0.146	0.036	0.518	+0.482	$(p=0.0005)$

† realized component count; the *static* count is 1 on this problem—see Section 8.

Table 4: Shared-resource join motif, $\beta = 0.3$. mc-q is the unbiased ground truth on this deterministic trace; ccf and LRQ flip sign (prefer the suboptimal action); s-ccf recovers the exact value.

scheme	$d_1(A - B)$	verdict
mc-q (ground truth)	-7.606	optimal action = B
LRQ	$+2.394$	FLIP $\rightarrow$ suboptimal
ccf	$+2.394$	FLIP $\rightarrow$ suboptimal
s-ccf	-7.606	unbiased, exact

Two cautions on reading Table 3. The intermediate gaps at $N=2$ and $N=4$ are *not* significant ($p = 0.56, 0.43$): the evidence for the decomposition is carried by $N=8$ and by multi-site, not by a fitted trend across four points. And the $\text{ccf} - \text{mc-q}$ column is flat until independence is genuinely present and then large, which is the qualitative shape Proposition 2 predicts, but we do not claim a scaling law from it.

Cost (Q2–Q3). Measured ccf/PPO wall-clock is $\approx 1.0\times$ across the sweeps (at the largest size ccf was in fact faster): the union-find and component return-to-go are negligible next to the GNN passes.

The ccf bias, exactly (Q5, Table 4). On a deterministic shared-resource join motif (decision d_1 : consume a shared resource now, delaying a second stream’s private reward, versus leave it free; $\beta = 0.3$), we compare each scheme’s $d_1(A-B)$ credit difference against mc-q’s, which is exact on a deterministic trace and hence the ground truth for which action is optimal.

ccf and LRQ not only lose magnitude, they reverse sign: both would select the suboptimal action. s-ccf recovers the ground truth exactly, on this and every instance of the motif we tested. On topologies with no shared-resource handoff (the N -copies and independent-site multi-site configurations above), realized and static partitions coincide exactly, so the ccf numbers reported for Q1–Q3 are precisely what s-ccf would have produced there; this equivalence does not extend to topologies with genuine handoffs, which is exactly what Table 4 isolates.

Table 5: Coupling truncation, real training scale (contested/joint topology, 30 epochs $\times$ 20 episodes/epoch, 12 seeds).

	minutes/cell	speedup	norm. final
CFPK without coupling truncation	36.25	—	1.000
CFPK with coupling truncation	26.53	1.37$\times$	1.000

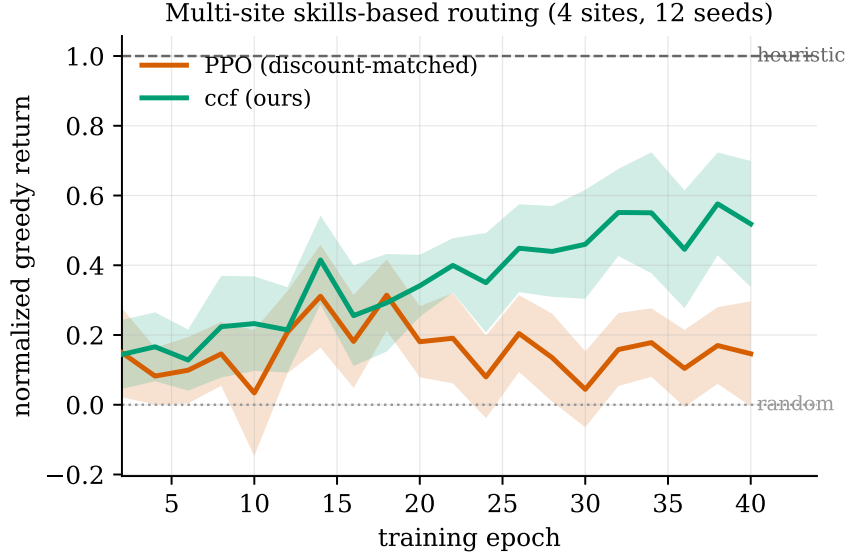


Figure 3: Realistic multi-site skills-based routing (12 seeds, 95% CI bands). PPO plateaus (does not learn the routing); ccf learns it and keeps rising.

Coupling truncation: exactness and cost (Q6). On a controlled net built to guarantee reconvergence after a corridor of decisions, coupling truncation reduces simulated env. step calls from 18 to 12 (33%) with paired gap/standard-error numerically *identical* with/without truncation, confirming Proposition 4’s exactness claim directly in the implementation. At real training scale, on a contested/joint topology chosen to fork often (full protocol: 30 epochs $\times$ 20 episodes/epoch, 12 seeds), coupling truncation gives a 1.37 $\times$ wall-clock speedup (36.25 $\rightarrow$ 26.53 minutes/cell) with `norm_final` *exactly identical* (1.000 vs. 1.000) across all 12 seeds (Table 5)—the zero-variance-contribution property of Proposition 4 predicts precisely this outcome: no change to what is learned, only to how it is computed.

7. The boundary-condition investigation

Corollary 1 makes a sharp prediction: s-ccf’s variance reduction scales with the number of causally independent components K , and collapses to *exactly* PPO when $K = 1$ (a single shared resource pool links every case into one component). This is the theorem’s own honest null, not a failure. We use a two-stage service-line environment with a single shared, skill-heterogeneous employee pool (henceforth the *bottleneck environment*) as the decisive $K=1$ case,

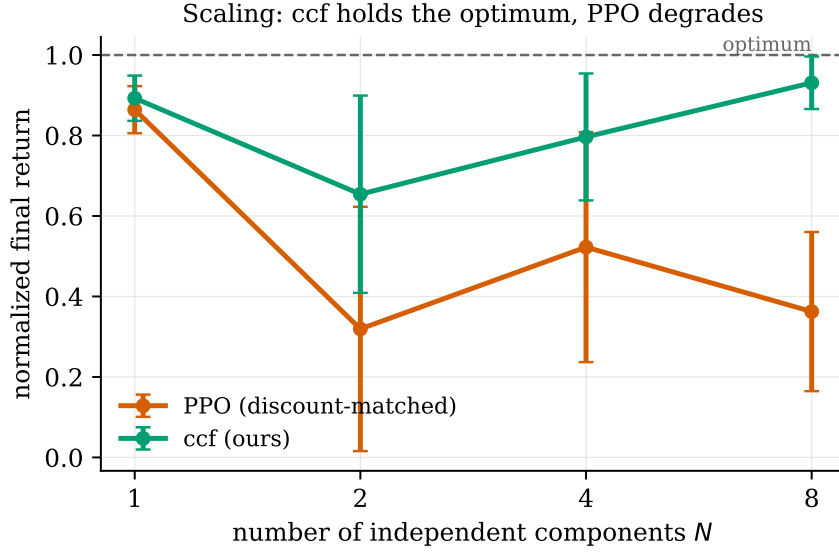


Figure 4: Scaling with the number of independent components N . ccf holds the optimum; PPO degrades and destabilizes. Tied at $N=1$ (the null).

and ask: can the residual gap on this genuinely single-component problem be closed by any other provenance- or structure-based route? We tested three independent, well-motivated candidates—never touching credit, only what the learner observes—and found each provably inert or empirically neutral. Table 6 summarizes; we report all three not as a coda but as a result, since two of the three failures follow from a proof rather than a correlation and together they localize the bottleneck’s difficulty precisely.

Reward shaping. Potential-based shaping from PN topology (Ng et al., 1999), SMDP-generalized, is provably policy-invariant for any potential function—verified directly in code via an exact terminal-value cancellation test, not merely cited. A full coefficient dial on the bottleneck environment (same cached PPO baseline reused across all four points) shows reach-rate cleanly monotonic in the injected-variance direction ($8/10 \rightarrow 4/10 \rightarrow 1/10$ seeds reaching 95% of the heuristic anchor as the coefficient rises $0.2 \rightarrow 0.5 \rightarrow 1.0$), and no coefficient produces a significant win: 0.739 ± 0.10 (baseline) vs. 0.766 ± 0.10 ($p = .48$, coefficient 0.2) vs. 0.420 ± 0.10 , 0W/10L (coefficient 1.0). The mechanism is that constant exogenous arrivals make the potential rise as well as fall, injecting a per-step signal on the same order of magnitude as the full episode return—unbiased in aggregate (guaranteed by the theorem) but a substantially noisier value-regression target at a finite budget.

Structural input features. Static, conflict-graph-derived scalars appended to each action node’s own input features (does this action type structurally compete for a token, and with how many; and, after a first null result, the minimum topological distance to the nearest reward among an action’s required inputs) are both statistically neutral ($p = .58$ and $p = .69$ respectively). The stronger finding is *why*: every action node already carries a one-hot action-type identity at zero hops, so the two competing action types in the bottleneck environment are already fully distinguishable with

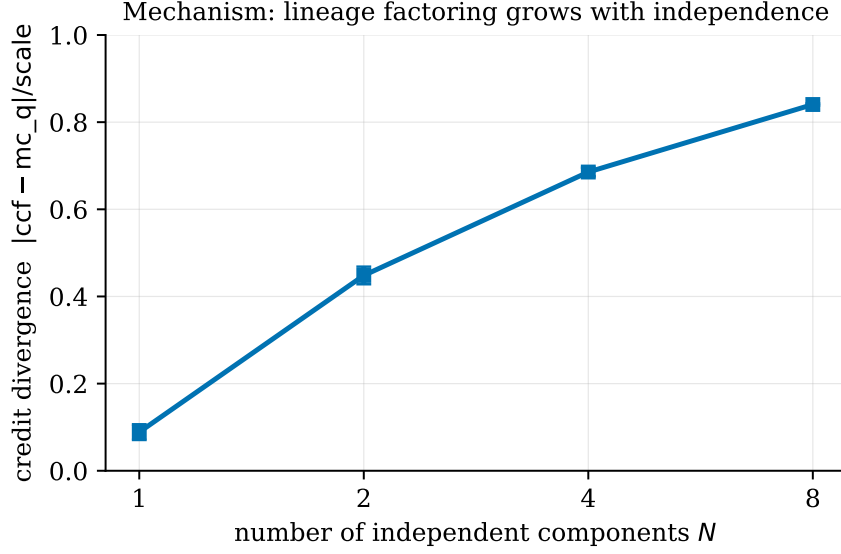


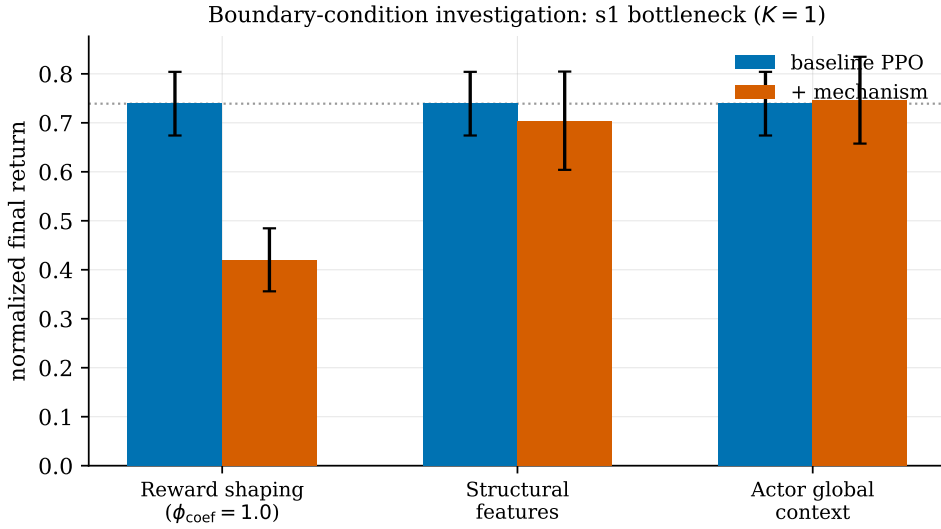
Figure 5: Mechanism: the divergence between ccf and PPO credit grows with the number of independent components, as Proposition 2 predicts.

or without any structural feature. Any scalar computed as a function of action type alone is therefore a deterministic function of information a single learned weight on the one-hot could already reconstruct—provably redundant, not merely observed to be unhelpful across two experiments, and true of this architecture on any topology, not only the bottleneck environment.

Actor architecture. We identify and prove a previously undocumented limitation: the actor decodes each action’s logit from only that action node’s own final message-passing embedding, unlike the critic, which has always pooled over all action/postpone node embeddings for its value estimate. Combined with graph edges directed strictly along token flow (no reverse edges at any real observation call site), an action’s logit can depend on state only via a forward path reaching it within the network’s depth. We prove this is a hard architectural fact, not a training-budget limitation: on a purpose-built topology of two structurally independent two-stage pipelines, perturbing one pipeline’s downstream marking left the other pipeline’s action logit *exactly, bit-for-bit unchanged* under a randomly initialized, untrained network ($\Delta = 0.000$)—reachability is a property of the computational graph, independent of learned weights. We fix this (concatenating the same pooled context the critic already computes onto each action node before decoding), confirm the fix closes the gap on the same proof topology ($\Delta = 0.000 \rightarrow 0.528$), and re-test on the bottleneck environment: norm. final 0.739 ± 0.10 (baseline) vs. 0.746 ± 0.14 (fixed architecture), $p = .91$ —no significant effect. The proof and the training result are not in tension: the bottleneck environment’s shared resource pool happens to recycle, giving the unfixed architecture a marginal, accidental backdoor to the same information, so the blind spot was never total there, and whatever fraction the fix newly exposes is not, in practice, decision-relevant enough to move a well-tuned baseline at this training budget.

Table 6: The boundary-condition investigation, bottleneck environment ($K=1$).

mechanism	theoretical status	result
Reward shaping	policy-invariant, any potential	neutral (0.2) to harmful (1.0), 0W/10L
Structural input features	mechanically correct	neutral, $p = .58 \rightarrow .69$; provably redundant
Actor global context	proven to close a real gap	neutral, $p = .91$; gap was already marginal here

Figure 6: Baseline PPO vs. each of the three boundary-condition mechanisms on the bottleneck environment ($K=1$; error bars are 95% CIs over seeds). None significantly improves on the baseline; reward shaping at $\phi_{\text{coef}}=1.0$ is significantly worse.

None of this is a failure of experimental design: each mechanism does exactly what it was built to do. It is evidence about where the bottleneck environment’s difficulty does *not* live—not in what the learner can observe, and not, per Corollary 1, in an avoidable credit bias on this genuinely single-component topology. We state this as an open problem rather than a solved one in Section 8.

8. Discussion

A decision rule, not just a result.. The causal-component count K (Proposition 2) is computed automatically, as a byproduct of the same union-find machinery the credit mechanism already runs, from a handful of simulated episodes under any reasonable policy—a concrete, pre-training diagnostic. If $K > 1$ reliably, the mechanisms in this paper apply, and the choice between them is a cost/granularity trade-off (below). If $K \approx 1$ essentially always—most commonly one shared, fully-utilized resource pool—neither mechanism will outperform a well-tuned PPO baseline, and Section 7 gives three independently verified reasons not to expect a workaround from adjacent directions either. Concretely: parallel production lines, multiple sites or service branches, and segmented (even loosely) resource pools are

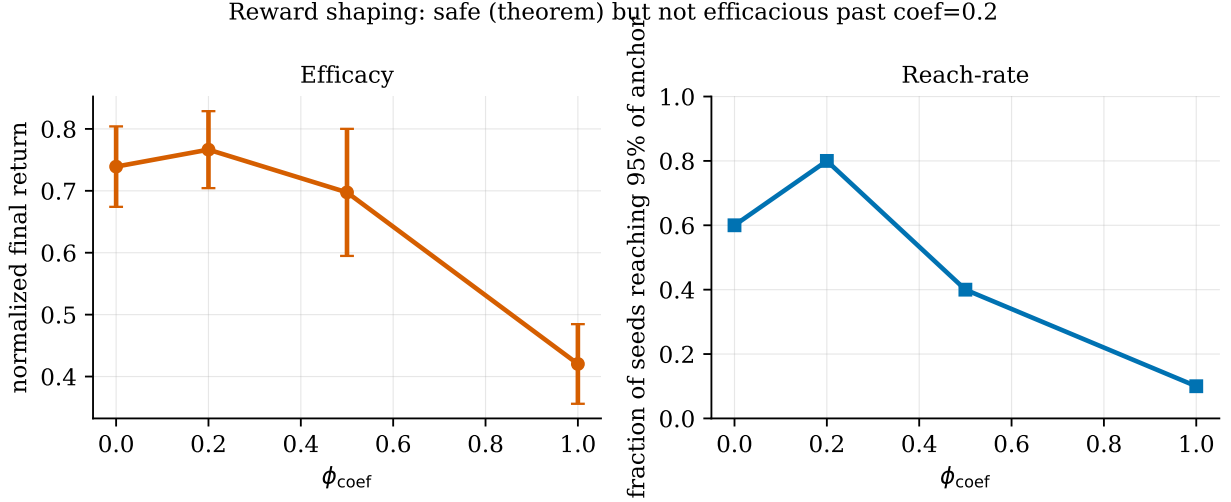


Figure 7: The reward-shaping coefficient dial on the bottleneck environment: efficacy (left) degrades once past $\phi_{\text{coef}} = 0.2$, and the fraction of seeds reaching 95% of the heuristic anchor (right) degrades monotonically—safety (Section 2, Ng et al., 1999) and efficacy are independent properties, and this mechanism has the first without the second at practical training budgets.

exactly the structures that produce $K > 1$; a single, centrally pooled team or machine bank serving one undifferentiated queue is exactly the structure that collapses to $K = 1$. Many real operations sit continuously in between, and the diagnostic handles this directly.

Two counts, and they select between the two variants.. K is not a single number: the diagnostic returns a *static* and a *realized* count, and the gap between them is what chooses s-ccf over ccf. K_{stat} partitions on topology—two decisions share a component if they *could ever* reach a common reward—so it is conservative and action-invariant, which is precisely the property that makes s-ccf unbiased (Section 5). K_{real} partitions on the realized provenance DAG, merging only decisions that *did* touch a common reward; it is finer, and it is what ccf uses.

The two can differ sharply, and they differ most on exactly the problems one cares about. On multi-site routing $K_{\text{stat}} = 1$ —a single shared pool of flexible generalists could in principle couple every site to every other—while $K_{\text{real}} = 12$, because in practice the sites are served locally and rarely interact. The same pattern holds on the scaled single-stage assignment problem ($K_{\text{stat}} = 1$, $K_{\text{real}} = 6$). Where the two agree, the variants coincide: on the N -copies family both equal N , and s-ccf and ccf are identical there.

The practical rule follows directly. **Compute both.** If $K_{\text{stat}} > 1$, use s-ccf: it is unbiased by construction and costs nothing extra. If $K_{\text{stat}} = 1$ but $K_{\text{real}} > 1$, s-ccf provably degenerates to PPO (Proposition 2, the $K=1$ case) and only ccf can act—at the cost of the action-dependent membership bias quantified in Table 4, which Section 4.1 shows is real but which we were unable to induce in *training* across four purpose-built configurations. If both are 1, neither applies. This is the honest form of the trade-off: s-ccf buys provable safety at the price of being inert on precisely the loosely-coupled problems that motivate the work, and ccf buys reach at the price of a bias whose practical bite we can

bound but not eliminate.

Choosing between s-ccf and cfpk. Both are safe by construction (Section 5), so the choice is a cost/granularity trade-off, not a correctness one. s-ccf is unbiased, requires no learned component beyond the standard critic, and costs a negligible amount of extra computation—a near-drop-in replacement for PPO’s baseline whenever $K > 1$; its cost is conservatism at AND-joins or shared handoffs, where its static partition can lump rarely-interacting components together. cfpk pays a real, bounded, partially reducible extra cost (Proposition 4) in exchange for exact per-decision credit that never needs to guess a partition, the right tool when s-ccf’s conservatism costs more than the extra compute would. The two are not mutually exclusive in principle: s-ccf as a low-cost default with cfpk’s exact replay invoked selectively at decisions flagged as synchronization-prone is a natural combination this paper does not evaluate.

What this paper does not solve. A genuinely single-bottleneck, fully-coupled problem gets no benefit from either mechanism, by the theory’s own honest prediction, and Section 7 closes off three plausible adjacent workarounds for that exact case with proofs, not single discouraging runs. A practitioner facing this shape of problem should not expect the family of methods this paper investigates to help, and should not spend engineering effort chasing them; plain, well-tuned PPO is already the right tool, and further gains likely require attacking exploration or value estimation under contention directly—a different problem than credit assignment.

Limitations and future work. Coupling truncation is one proven, free reduction in cfpk’s cost; it was not stacked with the mechanism’s other cost knobs (fork probability, lookahead depth, replication count), each trading cost against variance rather than bias—a combined ablation is the natural next step toward closing the remaining gap to plain-PPO cost. Neither mechanism depends on anything Petri-net-specific beyond an exact provenance record and, for cfpk, a forkable simulator; any executable model recording which past decisions caused which outcomes could apply the same machinery. We do not claim the single-component bottleneck is unsolvable, only that it is not an observability or credit-attribution problem as we tested those notions; exploration strategies tailored to contention, richer cross-decision value-function architectures, and hierarchical resource-allocation policies are directions this elimination motivates but does not test.

Managerial takeaway. For an operations manager weighing whether to invest in structure-aware RL over an off-the-shelf policy-gradient solver: the payoff scales with how independently the operation’s sub-streams can run, and this is checkable before committing to a training run. A multi-site or multi-line operation with loosely coupled sites or lines is where this investment pays off, safely and near-free (s-ccf) or at a bounded extra cost for exactness (cfpk). A centralized operation built around one shared, fully-utilized resource is not—and no amount of giving the learner more information about that shared resource changed that conclusion here, which is itself useful: the right response to a disappointing result on such a problem is not to keep feeding the model more structure, but to look elsewhere.

9. Conclusion

A-E PN records the causal provenance of every reward; existing DRL solvers discard it. We showed this structure supports two complementary, provably safe credit-assignment mechanisms—s-ccf, a near-free structural decomposition of the policy-gradient return, and cfpk, an exact forked-replay counterfactual credit with a provably lossless cost reduction—each dominating plain PPO exactly where the theory predicts it should and reducing to it exactly where the theory predicts it must. We then went further than either mechanism’s own validation and asked whether the honest null both predict on a genuinely single-component bottleneck could be closed by any other route that only changes what the learner observes, and showed, with proofs rather than omission, that three well-motivated candidates cannot. The result is not an unconditional method but a decision rule: provenance-based credit factoring pays off in proportion to how causally decomposable a dynamic task assignment problem actually is; where it does not decompose, the same formalism still affords an exact counterfactual by replaying the simulator, which is what cfpk does and why the two mechanisms cover complementary regimes. We now know, precisely rather than by assumption, where each payoff runs out.

References

- Arjona-Medina, J.A., Gillhofer, M., Widrich, M., Unterthiner, T., Brandstetter, J., Hochreiter, S., 2019. Rudder: Return decomposition for delayed rewards, in: NeurIPS.
- Benders, J.F., 1962. Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik* 4, 238–252.
- Bengio, Y., Lodi, A., Prouvost, A., 2021. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research* 290, 405–421.
- Dai, H., Khalil, E.B., Zhang, Y., Dilkina, B., Song, L., 2017. Learning combinatorial optimization algorithms over graphs, in: NeurIPS.
- Dantzig, G.B., Wolfe, P., 1960. Decomposition principle for linear programs. *Operations Research* 8, 101–111.
- Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., Whiteson, S., 2018. Counterfactual multi-agent policy gradients, in: AAAI.
- Harutyunyan, A., Dabney, W., Mesnard, T., et al., 2019. Hindsight credit assignment, in: NeurIPS.
- Hu, Z., Dong, Y., Wang, K., Sun, Y., 2020. Heterogeneous graph transformer, in: WWW, pp. 2704–2710.
- Jordan, W.C., Graves, S.C., 1995. Principles on the benefits of manufacturing process flexibility. *Management Science* 41, 577–594.

- Lo Bianco, R., Dijkman, R., Nuijten, W., van Jaarsveld, W., 2023. Action-evolution petri nets: a framework for modeling and solving dynamic task assignment problems. arXiv preprint arXiv:2306.02910 .
- Lo Bianco, R., et al., 2025a. Gympn: A library for decision-making in process management systems. arXiv preprint arXiv:2506.20404 .
- Lo Bianco, R., et al., 2025b. A universal approach to feature representation in dynamic task assignment problems. arXiv preprint arXiv:2507.03579 .
- Mazyavkina, N., Sviridov, S., Ivanov, S., Burnaev, E., 2021. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research* 134, 105400.
- Mesnard, T., Weber, T., Viola, F., et al., 2021. Counterfactual credit assignment in model-free reinforcement learning, in: ICML.
- Nazari, M., Oroojlooy, A., Snyder, L., Takáč, M., 2018. Reinforcement learning for solving the vehicle routing problem, in: NeurIPS.
- Ng, A.Y., Harada, D., Russell, S., 1999. Policy invariance under reward transformations: Theory and application to reward shaping, in: ICML, pp. 278–287.
- Rashid, T., Samvelyan, M., de Witt, C.S., Farquhar, G., Foerster, J., Whiteson, S., 2018. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning, in: ICML.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P., 2016. High-dimensional continuous control using generalized advantage estimation .
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 .
- Sunehag, P., et al., 2018. Value-decomposition networks for cooperative multi-agent learning, in: AAMAS.
- Sutton, R.S., McAllester, D., Singh, S., Mansour, Y., 2000. Policy gradient methods for reinforcement learning with function approximation .
- Wolpert, D.H., Tumer, K., 2001. Optimal payoff functions for members of collectives. *Advances in Complex Systems* 4, 265–279.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C., 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning, in: NeurIPS.

Appendix A. Additional remarks on s-CCF and coupling truncation

Remark 1 (Contrast with LRQ). LRQ restricts credit to a single token lineage, dropping the rewards that sibling lineages—the roads not taken at d —would have earned. Those are causal descendants of alternative actions at d and are not mean-independent of a_d ; dropping them violates the analogue of Assumption 1 and biases the estimate. CCF/s-CCF keep the whole component, so they never drop an a_d -dependent reward.

Remark 2 (Shared RNG). With a single global random stream, changing a_d can shift the order in which cross-component events consume random draws, so their realized values may differ across counterfactual actions. This affects realized Δ_d but not $\mathbb{E}[\Delta_d \mid \mathcal{F}_d, a_d]$, so Assumption 1 is unaffected.

Remark 3 (No RNG-stream alignment required for coupling truncation). A naive implementation might assume the two forked branches' random-number generators are in the same position when they reconverge in state—false in general, since the branches take a different number of intervening exogenous events to reach s^* . Proposition 4's proof never uses this: it only uses that s^* is the same state for both (Assumption 3), and that one draw from that state is reused for both rather than two draws being taken.

Remark 4 (Agnosticism to the tail procedure). Proposition 4 only uses that $\tilde{V}(s^*)$ is generated by a fixed procedure applied to s^* alone; it does not matter whether that procedure is pure Monte Carlo simulation to episode end or a value-tail-bootstrapped truncation at a finite lookahead, so the proposition covers CFPK's production configuration without modification.

Remark 5 (Scope). Coupling truncation is gated off for the lineage-credit suffix: a post-hoc lineage analysis is defined per complete causal trace, and sharing one tail between two branches' separate traces has no well-defined per-branch lineage meaning. Proposition 4 applies to the plain (non-lineage) suffix, which is what CFPK uses in production.